

Statistical Foundations of Machine Translation and Large Language Models

Taeyun Kim
Mentor: Tony Lei
SPA Directed Reading Program

Winter 2026

Introduction

I worked on the topic “Statistical Foundations of Machine Translation and Large Language Models” with mentor Tony Lei. In this report, I give a high-level summary of the subject, tracing the progression from classical probabilistic language models through to the Transformer architecture that underlies modern large language models (LLMs).

1 Language Models and Tokenization

The goal of a language model is to assign a probability to a sequence of words. Given a vocabulary $\mathcal{V}$, a language model defines:

$$P(W_1, W_2, \dots, W_n)$$

and is used to answer: given some history of previous words, what is the most likely next word?

Before a model can process text, it must be converted into discrete units called **tokens**. **Tokenization** is the process of splitting raw text into these units. Common strategies include word-level, character-level, and subword tokenization. The most widely used subword method is **Byte Pair Encoding (BPE)**, which starts from individual characters and iteratively merges the most frequent adjacent pair until a target vocabulary size is reached. BPE handles rare words gracefully by decomposing them into recognizable subpieces, and is the tokenizer behind most modern LLMs.

2 N-gram Models and the Chain Rule

2.1 Chain Rule Decomposition

The joint probability of a sequence can be decomposed exactly using the chain rule of probability:

$$P(W_1, W_2, \dots, W_n) = P(W_1) \prod_{k=2}^n P(W_k \mid W_1, \dots, W_{k-1}).$$

Each word depends on its entire preceding context. While exact, this is computationally heavy for long sequences as the number of possible histories grows exponentially.

2.2 The Markov Assumption and N-grams

To make the problem tractable, we invoke the **Markov assumption**: the probability of the next word depends only on the previous $n - 1$ words,

$$P(W_k | W_1, \dots, W_{k-1}) \approx P(W_k | W_{k-n+1}, \dots, W_{k-1}).$$

This gives rise to the family of **N-gram models**:

Model	Order	Context
Unigram	$n = 1$	None
Bigram	$n = 2$	Previous word
Trigram	$n = 3$	Previous two words

N-gram probabilities are estimated from corpus counts via **Maximum Likelihood Estimation (MLE)**. For the bigram case:

$$P(W_k | W_{k-1}) = \frac{C(W_{k-1}, W_k)}{C(W_{k-1})}$$

where $C(\cdot)$ denotes the count of a sequence in the training corpus. For example, if *cat jumped* appears 5 times and *cat* appears 20 times, then $P(\text{jumped} | \text{cat}) = 5/20 = 0.25$.

2.3 Perplexity

Perplexity is the standard metric for evaluating a language model. It is defined as the inverse geometric mean per-token probability on a test corpus of N tokens:

$$\text{PP}(W) = P(W_1, W_2, \dots, W_N)^{-1/N}.$$

Intuitively, perplexity measures how “surprised” the model is by held-out text. Lower perplexity indicates a better model. A model with perplexity k is roughly as uncertain at each step as if it were choosing uniformly among k alternatives.

3 Word Embeddings

N-gram models treat words as atomic symbols with no notion of similarity. **Word embeddings** address this by mapping each token $w \in \mathcal{V}$ to a dense vector $\mathbf{v}_w \in \mathbb{R}^d$, so that semantically similar words have geometrically similar vectors. Two standard similarity measures are:

Dot product:

$$\mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^d u_i v_i$$

Cosine similarity:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} = \cos \theta$$

Cosine similarity is preferred when vector magnitude is not meaningful, as it captures only directional alignment. Embeddings allow models to generalize across related words and serve as the input layer to all modern neural language models.

4 Transformers and the Attention Mechanism

4.1 Architecture Overview

The **Transformer** is the neural architecture underlying modern LLMs. Unlike recurrent models, Transformers process all tokens in parallel, enabling efficient computation and long-range dependency capture. The processing pipeline is:

Sentence $\rightarrow$ Token IDs $\rightarrow$ Embeddings $\rightarrow$ Positional Encoding $\rightarrow$ Attention Layers $\rightarrow$ Output.

Positional encoding is added to the embeddings to inject token-order information, since the attention operation itself is permutation-invariant.

4.2 Scaled Dot-Product Attention

The core operation is **scaled dot-product attention**. Given an input matrix $X \in \mathbb{R}^{N \times d}$, three matrices are produced via learned linear projections:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

where $W^Q, W^K, W^V \in \mathbb{R}^{d \times d_k}$ are learned weight matrices. The **Query** Q encodes “what am I looking for?”, the **Key** K encodes “what do I contain?”, and the **Value** V encodes “what do I pass along?”

The attention output is:

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

The product $QK^\top \in \mathbb{R}^{N \times N}$ yields pairwise similarity scores between every pair of tokens. Dividing by $\sqrt{d_k}$ prevents the scores from growing too large and destabilizing the softmax. The softmax normalizes each row into a probability distribution, and multiplying by V produces a weighted combination of value vectors — a contextually-aware representation for each token.

Causal masking: In autoregressive LLMs, entries above the diagonal of $QK^\top$ are set to $-\infty$ before softmax, ensuring each token can only attend to itself and earlier tokens.

4.3 Autoregressive Generation and Temperature

Modern LLMs generate text token by token. At each step t , the next token is selected either by **greedy decoding**,

$$W_t = \arg \max_w P(W = w \mid W_{<t}),$$

or by sampling from the distribution. **Temperature scaling** modulates this by dividing logits by $\tau > 0$ before softmax:

$$y = \text{softmax}(u/\tau).$$

As $\tau \rightarrow 0$ the output becomes deterministic; as τ increases outputs become more diverse and random.

Conclusion

The statistical foundations of modern language models form a coherent progression. N-gram models establish the probabilistic framework via the chain rule and the Markov assumption, MLE estimates parameters from data, and perplexity provides a meaningful evaluation metric. Word embeddings replace discrete symbols with continuous geometric representations. Finally, the Transformer with scaled dot-product attention scales these ideas to massive corpora, capturing contextual dependencies that classical models cannot. Together, these components explain how a model moves from raw text to coherent, fluent generation.

Reflection

This topic turned out to be more math-heavy than I expected. I thought I had a good understanding of probability and statistics from my coursework, but working through chain rule decomposition and the attention mechanism showed me there was a lot more depth to it than I initially realized.

The most surprising part was seeing how much of modern AI is really just statistics under the hood. Before this, I think I had a vague impression that systems like ChatGPT were doing something almost mysterious. But tracing the ideas from N-gram models all the way through to Transformers made it clear that everything connects back to concepts like probability distributions and maximum likelihood, which I had already seen in class, just applied in ways I was not aware of.

It was a good reminder that the statistics and math we learn is not just abstract exercises. It actually shows up at the core of some of the most interesting technology out there right now.

References

- [1] Jurafsky, D., & Martin, J. H. (2026). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models*.